

KeyQuorum

Manual

Bailey Forbes

A manual for the KeyQuorum hardware-key quorum file-sharing system: hardware keys and devices, split-tree quorum protection, private sign bridges, authenticated updates, moving identities between devices, sending files directly, tracked files with their own signed history, and the mailbox relay that connects machines that are not open together.

Contents

1 Overview	1
1.1 Concept	1
1.2 What this manual covers	1
1.3 Status	1
1.4 Browser lab	1
2 Getting started	2
2.1 Building the CLI	2
2.2 The local database	2
2.3 Two kinds of identity	2
3 Hardware keys	2
4 Devices and custody	3
4.1 Presenting a slot	3
4.2 Custody modes	4
4.3 Parent approval	4
5 Splitting a secret	4
5.1 A two-department example	5
5.2 Growing and rebinding a tree	5
5.3 Bridges (whitelist and pairing)	5
5.4 Publishing and fetching topology	6
6 Hierarchical principals	6
6.1 What a hierarchical principal is	6
6.2 Four dimensions, four owners	6
6.3 A worked example	7
7 Quorum-protected files	8
7.1 Expiry	8
7.2 Verbose unlock	8
8 Password-protected files and credentials	8
9 Signatures, export, and sharing	9
10 Private sign bridges	9
10.1 Envelope shape	10
10.2 Revocation and rotation	11
10.3 Evicting a hardware key	11
11 Authenticated updates: reissue and restructure	11
11.1 The four-part authorization rule	11

11.2	What a reissue changes	12
11.3	Restructure vs. publish	12
12	Moving a key between devices	13
12.1	Package format and audit	13
12.2	Transferring through the mailbox	13
13	Sending a file	14
14	Tracked files and history	15
14.1	What a tracked file holds	15
14.2	Trust: who may change a file	16
14.3	Sharing falls back to the last trusted revision	17
14.4	Sending and receiving a tracked file from the command line	18
14.5	Copies, forks and merging	20
14.6	Terms for proposing a change	21
14.7	Asking for a file, or for a change	21
14.8	History snapshots and the index	22
14.9	Expiry and tombstones	23
14.10	Signed events	23
14.11	Recording what happens at a gate	23
14.12	Limits this version states	23
15	The mailbox relay	24
15.1	The device mailbox	24
15.2	Recovering from a lost key or a missed pull	25
16	Security model	25
16.1	What the mailbox can and cannot see	25
16.2	Custody, quorum, and delegation, at a glance	25
16.3	Envelopes and authenticity	25
17	File formats quick reference	26
17.1	Roadmap	26
18	Command reference	26
18.1	Identity and hardware keys	27
18.2	Split-key trees	27
18.3	Password vault	27
18.4	Bridges	28
18.5	Authenticated updates	28
18.6	Files and credentials	28
18.7	Signing	29
18.8	Delivery and the mailbox relay	29
18.9	Tracked files	29

18.10 Device transfer	31
18.11 <code>keyquorum-device</code> : the container tool	31
19 Contributing	31
20 License	32

1 Overview

KeyQuorum is a secure file-sharing system centered on hardware key sharing. Files are encrypted and bound to registered physical tokens, such as USB devices. Access requires the necessary hardware keys to be presented before a protected file can be unlocked, providing layered, hardware-backed access control.

Note

This physical binding depends on a key actually being placed in a `keyquorum-device` container (Section 4). An unplaced `-share-file` key file is classified by its own key identity: two such files can both count toward `minimum_physical_devices` even while sitting on the same physical drive, since nothing ties either one to a container's `device_id`.

1.1 Concept

- A **key** — the data key protecting a file, or a secret split for its own sake — can be divided recursively: split into parts, any of which can itself be split further, forming a tree. A company master key can split across departments, and one department's own share can split again across its team. A flat “*M*-of-*N* hardware keys” quorum is just the simplest one-level tree.
- **Unlocking** a protected file means reconstructing its key's tree: presenting a quorum of the registered keys, rather than a single token or password.
- The goal is layered, hardware-backed access control that resists single-point compromise. A lost or stolen key alone should not be enough to unlock protected data.

1.2 What this manual covers

This manual documents the `keyquorum` and `keyquorum-device` command-line tools: generating and registering hardware keys, building and reconstructing quorum trees, protecting files and credentials, private signing bridges, authenticated updates to a live tree, moving identities between physical devices, sending files directly to another person, tracked files that carry their own signed revision history, and the mailbox relay that carries all of the above between machines that are not open at the same time.

Note

Passwords, PINs, and hardware-key material are always prompted for interactively, or read from a `-share-file` key path, rather than taken as plain command-line arguments. This keeps secrets out of shell history and process listings.

1.3 Status

The CLI covers most of what this manual describes. Hardware-key quorum splitting and reconstruction are implemented in software: a key file with no container placement is still one device, several identities can share one container, a non-root tree restructure waits for its parent to countersign, and `transfer copy` / `transfer move` carry an active identity to a second device that is open at the same time, or through the mailbox as a sealed letter when it is not.

Tracked files (section 14) are implemented through `keyquorum file`; that section ends with what their design still plans. Two other things remain outside the current scope: persisted private-key custody beyond a key file or a `keyquorum-device` container (for example an OS keychain), and `import` of password-vault or locked-file `export` bundles (the bundle format itself is final; private-bridge `.kqpb` import is already implemented).

1.4 Browser lab

KeyQuorum Lab runs this crate in a browser as a small sandboxed machine, compiled to WebAssembly. It has its own files, a SQLite store per person, mock USB drives, and a relay answering in the page. Every button and every terminal line runs the real `keyquorum` and `keyquorum-device` commands, so the lab demonstrates the same behavior described in this manual. It is itself a browser-based virtual machine that emulates USB-token custody entirely in

memory, with published default passphrases — so it is not physical hardware-backed security. It is a good place to try the command sequences in this manual without provisioning real hardware.

Note

The lab’s own GUI — its tabs, the Active user bar, and its guided in-browser tutorials — is documented separately in the *KeyQuorum Lab* manual, not in this one. This manual covers the `keyquorum` / `keyquorum-device` CLI itself, which the lab runs underneath its interface unchanged.

2 Getting started

2.1 Building the CLI

Build the customer CLI with a release build; the binary is `target/release/keyquorum`.

```
cargo build --release
```

Adding `-features provider` compiles mailbox-host capabilities into the same crate. That flag is a build capability, not authorization: official clients trust a relay only when it presents a KeyQuorum-signed provider certificate and proves possession of the matching private key. This manual covers the customer-facing surface — the URL and API key a provider gives you, and the `loadkey` / `relay push` / `relay pull` commands built on top of them. The rest of this manual explains what each area of the CLI is for and how the pieces fit together; the Command reference at the end is the complete index of every `keyquorum` and `keyquorum-device` command and subcommand. `keyquorum <command> -help` still has the full, current flag list for any one command.

2.2 The local database

Most commands read and write a local SQLite database that holds your split trees, registered keys, and (for a relay-connected identity) a sealed copy of your mailbox bearer. Pass `-db` to point at a specific file; when omitted, commands use a default database in the current directory. Running the same commands against different `-db` files is how one machine can hold several independent identities (for example `alice.sqlite` and `bob.sqlite` in the walkthroughs throughout this manual).

Security

Never commit a SQLite database, a private key file, an API bearer, a provider certificate, a revocation list, a device descriptor (`device.kq`, `device.skey`), or a slot token (`*.kqst`) to version control. See `.gitignore` for the patterns this repository already excludes.

2.3 Two kinds of identity

KeyQuorum draws a line between two things that both use the word “key”:

- A **hardware key** is a registered Ed25519/X25519 keypair, generated with `keyquorum generate` or brought in with `keyquorum register`. It is what a person, device, or slot presents to satisfy a quorum, sign a message, or open a sealed envelope.
- A **split key** (or “key” in `split` / `tree` output) is the secret being protected — a standalone escrow, or the data key encrypting a specific file. It lives as a tree of leaves, each leaf sealed to one hardware key.

The rest of this manual moves through the CLI roughly in the order you would use it for a new deployment: registering hardware keys and devices, building a split tree, protecting files and credentials against it, then the supporting machinery — private sign bridges, authenticated updates, device transfer, direct file delivery, and the mailbox relay that connects machines that are not open together.

3 Hardware keys

Hardware and tree verbs are top-level commands (`generate`, `split`, `bind`, and so on).

```
# Generate a keypair. The private key is printed to stdout ONCE and never
# written to disk by this tool -- redirect it yourself. Since the shell,
# not keyquorum, creates that redirected file, set a restrictive umask
# first so it is not created world- or group-readable.
umask 077
keyquorum generate --type encryption --public-key-out alice.pub \
  --label alice --register > alice.key
keyquorum register --type encryption --label bob --public-key-file bob.pub
keyquorum list
```

Security

A file created by shell redirection (`> alice.key`) gets whatever permissions your shell's umask allows — `keyquorum` never writes to it and cannot restrict it after the fact. Always set a restrictive umask (or move the file into owner-only permissions yourself) before redirecting a generated private key to disk. This applies to every `generate ... > key-file` line in this manual, not only this one.

`generate -register` writes the public key and records it in one step. `register` brings in a public key that already exists elsewhere (a PEM or OpenSSH file, or another person's `.pub`) under a chosen label. `list` shows every registered hardware key and every live split tree.

Note

Two key types are used throughout this manual: `-type encryption` keys unwrap sealed shares and envelopes; `-type signing` keys produce and verify Ed25519 signatures (private bridges, `sign` / `verify`, authenticated updates). A label commonly needs both — an encryption key to receive things, and a signing key to authorize or countersign them.

4 Devices and custody

A key file with no container placement is its own device. Distinct `-share-file` keys count as distinct physical devices, including when `minimum_physical_devices` is greater than 1. Presenting the same key twice still counts once.

`keyquorum-device` manages the container directory. `keyquorum device` registers a slot's public key in the org store and binds it to the opened container. Several identities can live as passphrase-wrapped slots in one directory (a mounted USB, or a stand-in). The directory is an index: `device.kq` (KQDV v2) is signed by `device.skey`, and each slot token (`*.kqst`, KQST) seals that same device id. Opening a slot checks the token against the descriptor, so a rewritten descriptor id cannot make one container count as two devices. Path, mount, and volume label are *not* the device identity — `relocate` moves a slot's keypair onto another container, and that is the authenticated change of `device_id`.

```
keyquorum-device init ./usb
keyquorum-device provision ./usb --label M.S
keyquorum-device list ./usb
keyquorum device register ./usb --slot M.S --type encryption
keyquorum device bind ./usb --slot M.S
keyquorum-device relocate --from ./usb --to ./usb2 --label M.S
```

`register` records the public key; a lone key file still uses `register` and `-share-file`. `bind` writes the placement from the container this process opened, tying that key to the container's `device_id`.

4.1 Presenting a slot

Commands that unwrap a key also take `-slot container=label` beside `-share-file`. This includes `reconstruct`, `add`, `bind`, `revoke`, an access quorum `unlock`, `bridge private import`, `sign`, `remove-member`, and `relay pull`. `tree countersign` takes `-device` and `-slot` instead.

```
keyquorum reconstruct <key-id> --slot ./usb=M.S --slot ./usb2=M.A \
  --output master.pub
```

```
keyquorum --db alice.sqlite relay pull --import --slot ./usb=M.S
```

4.2 Custody modes

split, tree, and access quorum -state 0 set -custody (hardware or logical) and -minimum-physical-devices.

- **Hardware** custody requires each share Shamir actually consumes to sit on its own device id.
- **Logical** custody lets several slots on one container meet that Shamir threshold together; the container still counts as one device toward the minimum. Logical mode is for development and constrained hardware — logical slots are not a hardware quorum.

Reconstruction searches threshold-sized subsets for one that meets `minimum_physical_devices`. Unused extra shares are not counted.

```
keyquorum split --label master --threshold 2 --custody logical \
--minimum-physical-devices 2 \
--leaf M.S=SoftwareDepartment.pub --leaf M.A=AccountingDepartment.pub \
--source master.pub --generate-keys --register
keyquorum tree <key-id> --custody hardware --minimum-physical-devices 2
```

4.3 Parent approval

unlock_approval is none (the default) or parent. Parent approval is opt-in and sits on top of both Shamir and the device count: each leaf that contributed a share needs its direct parent's signature.

```
# The leaf's own key, and a registered *signing* key for its direct
# parent M.S -- the approval check verifies against that registration.
keyquorum generate --type encryption --label M.S.1 --register \
--public-key-out m-s-1.pub > m-s-1.key
keyquorum generate --type signing --label M.S --register \
--public-key-out M.S.sign.pub > M.S.sign.key

keyquorum access quorum --state 0 --source ./secret.txt \
--encrypted-path ./secret.txt.kqenc \
--leaf M.S.1=m-s-1.pub --threshold 1 --unlock-approval parent
keyquorum access quorum --state 1 --id <file-id> --share-file m-s-1.key \
--approve "M.S.1=M.S.sign.key"
```

-approve takes either leaf=signing-key-file or leaf=container>slot.

5 Splitting a secret

Splitting is how a standalone secret is escrowed, and how a file's data key is protected — both use the same tree machinery.

Note

The live SQLite tree *is* the spec for whoever holds that store. An operator or publisher may keep the full org tree; a personal store should hold only the nodes that person needs for signing and access control: their lineage, their descendants, siblings of their node, and the fixpoint of *established* bridge peers (the peer and the peer's ancestors, not the peer's unrelated siblings). Pushing from a personal store merges that subgraph into the relay; it does not replace the canonical document. There is no JSON file to author by hand — `split`, `bind`, `add`, `revoke`, `bridge`, and `access quorum -state 0 -leaf` write the tree in place. `tree -output` writes a snapshot of whatever is stored now; `-tree-spec FILE` remains only for a nested one-shot tree.

Labels must be unique within a tree. `tree <id> -node A B` prints that set's lowest common ancestor; `reconstruct -node A B` starts recovery at that ancestor. `tree` with no id lists every stored spec.

5.1 A two-department example

`split` `-source` `master.pub` escrows that key file (hex, PEM, or OpenSSH). `-leaf` builds a one-level tree and binds every sibling pair, so `M.S <-> M.A` is recorded even after a later refresh. Reconstruct with the holders' key files and `-output`. A 2-of-2 department tree (`M = master.pub`, `M.S = SoftwareDepartment.pub`, `M.A = AccountingDepartment.pub`) reassembles the master from the two department keys:

```
keyquorum split --label master --threshold 2 \
  --leaf M.S=SoftwareDepartment.pub --leaf M.A=AccountingDepartment.pub \
  --source master.pub --generate-keys --register
keyquorum reconstruct <key-id> \
  --share-file SoftwareDepartment.pub --share-file AccountingDepartment.pub \
  --output master.pub
keyquorum tree
keyquorum tree <key-id> --output org.json
```

`-generate-keys` creates each leaf's `.pub` / `.key` pair; `-register` records those public keys. Dotted leaf labels (`M.S`, `M.A`) infer the root node `M`. `-bind M.S=M.A` adds an extra pairing; `-leaf` already binds sibling leaves. A `.pub` share file uses the sibling `.key` (or a private-key prompt) to unwrap that department's sealed share — both departments are required because `M`'s threshold is 2.

5.2 Growing and rebinding a tree

Later commands keep changing the same tree. Pairings are stored by node id, so they survive a secret refresh, a new leaf, or a leaf moving to a new public key:

```
# Pair two nodes (whitelist both ways + establish the link)
keyquorum bind <key-id> --node M.S --peer M.A

# Move M.S onto a new token; node id -- and the bind -- stay put
keyquorum bind <key-id> --node M.S \
  --public-key-file NewSoftware.pub --share-file SoftwareDepartment.key \
  --register

# Grow M to 2-of-3; survivor node ids (and M.S <-> M.A) stay put
keyquorum add <key-id> --parent M --node M.F \
  --public-key-file FinanceDepartment.pub \
  --share-file SoftwareDepartment.pub --share-file AccountingDepartment.pub \
  --generate-keys --register
```

5.3 Bridges (whitelist and pairing)

```
keyquorum tree 1 --node alice bob
keyquorum reconstruct 1 --node alice bob --share-file alice.pub \
  --share-file bob.key
keyquorum bridge allow 1 --node alice --peer it
keyquorum bridge add 1 --from alice --to it
keyquorum bridge list 1
keyquorum bridge remove 1 --from alice --to it
keyquorum bridge deny 1 --node alice --peer it
```

`bind -peer` is the usual way to stand up `A <-> B`. `bridge allow` / `deny` change the whitelist only. `bridge add` / `remove` stand up or tear down an established pairing — `add` requires a whitelist hit on either side; `deny` also drops any pairing.

Note

This bridge/whitelist pairing is unrelated to a *private sign bridge* (Section 10), which is a separate, N-person co-signing group.

5.4 Publishing and fetching topology

The relay stores the **full** public tree as a JSON document. Sending data (**relay push**, including private-bridge envelopes) merges the sender's public topology into that document; nodes the sender does not hold stay in place, so a personal subgraph cannot replace canonical context. Fetching data (**relay pull**) returns the slice this device's encryption fingerprint is allowed to see, and the CLI merges it into local SQLite before importing envelopes. A later push that adds a bridge such as **M.S.2** <-> **M.A.1** expands the next pull for **M.S.2** to include **M.A.1** as topology-only (no sealed share).

tree fetch refreshes topology without downloading envelopes. **tree publish** remains if you need to replace the document without an envelope.

```
# Person: envelopes plus the slice this pull key is allowed to see
keyquorum --db alice.sqlite relay pull --import --share-file alice.key

# Topology only (no envelopes), including first fetch onto an empty store:
keyquorum tree fetch <key-id>
keyquorum tree fetch --label master
```

6 Hierarchical principals

KeyQuorum implements a hierarchical cryptographic principal model in which organizational ancestry, delegated authority, threshold key reconstruction, and physical key custody are represented as separate but interacting security dimensions. The dotted labels seen throughout this manual — **M**, **M.S**, **M.S.1** — are not just a naming convention. Each one is a *principal*: an entity the system can recognize, place in an ancestry tree, and reason about independently along each of those four dimensions. This section makes that model explicit, because it is what a lot of the behavior in Sections 4, 10, and 11 is actually enforcing.

6.1 What a hierarchical principal is

In access-control and cryptographic-key-management literature, a *hierarchical principal* scheme is any design where identities are arranged in a tree and position in that tree carries meaning: an ancestor has some form of standing over its descendants that a sibling or an unrelated node does not. Familiar examples differ sharply in *what* that standing lets an ancestor do:

- In a **PKI certificate chain**, the hierarchy *is* the trust chain, and a parent's compromise is total: whoever holds a CA's private key can mint a valid certificate for any descendant, so the hierarchy is really one long single point of failure wearing a tree shape.
- In **Hierarchical Identity-Based Encryption (HIBE)**, the hierarchy is a *key-derivation function*: a parent's master secret can compute every descendant's private key directly. Delegation and key escrow are the same operation.
- In **Kerberos realms or Unix group trees**, the hierarchy governs *grants* (who is allowed to do what) but carries no cryptographic material at all.

KeyQuorum's dotted-label tree is closer to the third case than the first two, and that is a deliberate design choice, not an omission. An ancestor label such as **M** or **M.S** can *authorize* specific, narrowly-scoped changes about a descendant — but it can never *derive*, *recover*, or *stand in for* that descendant's private key material. Compromising **M**'s signing key lets an attacker forge authorization letters; it does not, by itself, decrypt anything **M.S.1** holds.

6.2 Four dimensions, four owners

The tree of dotted labels is the substrate all four dimensions are indexed by, but each dimension is a distinct *access structure* — in the secret-sharing sense of the term, a rule for which sets of conditions are authorized — enforced by its own code and its own invariants:

1. **Organizational ancestry.** A pure naming relation: is **M.S** an ancestor of, or equal to, **M.S.2**? This question is answered by string structure alone (dotted-label prefixes), independent of what secret, signature, or device sits at either node. It is the one piece of the model that every other dimension is built on top of.

2. **Delegated authority.** A signature capability: which *registered signing keys* may authorize a change addressed to a given label? The rule (Section 11) is that the signer must be the subject itself, or a dotted-label ancestor whose signing key this store already has on file. A restructure proposed by a non-root label waits for its direct parent’s countersignature; a direct tree update attempted by that same authorizer instead of a proposal is refused outright; and a routine employee reissue by the direct parent stays a single signature. Authority here is the power to *authorize a documented change*, checked fresh against a signature every time — it is never the power to read or reconstruct what the change is about.
3. **Threshold key reconstruction.** A purely mathematical property of the Shamir split tree (Section 7): an M -of- N subset of *sibling* leaf shares reconstructs the secret those siblings were split from. This has nothing to do with ancestry or authority — reconstructing M ’s secret from $M.S$ and $M.A$ ’s shares does not require M ’s signature or cooperation, and M cannot reconstruct it alone by virtue of being the parent label. `unlock_approval = parent` is an optional, additional layer on top of this dimension, not a substitute for it: Shamir’s threshold must still be met, *and* each contributing leaf’s direct parent must sign off.
4. **Physical custody.** A count of distinct hardware devices (Section 4): how many separate device ids back the shares actually consumed by a reconstruction, independent of where those devices’ labels sit in the tree. Two slots on one USB drive count as one device no matter how far apart their labels are in the hierarchy; two key files on two different drives count as two devices even if one label is the other’s direct parent.

This split is visible in the codebase’s own module boundaries, not just in this manual’s organization.

- `private_bridge.rs` owns the dotted-label ancestry test (`is_ancestor_or_self`, `parent_node_label`).
- `org_update.rs` and `authority.rs` own delegated authority.
- The Shamir math for threshold reconstruction lives with the split tree itself.
- `keys.rs` and `device.rs` own the hardware-key registry and physical-device bookkeeping.

A change to one dimension should not require touching the others.

6.3 A worked example

Take $M.S.2$, three labels deep under root M via $M.S$. The four dimensions give four independent, simultaneous answers about that one principal:

Dimension	Answer for $M.S.2$
Ancestry	$M.S$ and M are ancestors; a sibling such as $M.S.3$ is not.
Delegated authority	$M.S.2$ itself, $M.S$, or M may sign a reissue naming $M.S.2$ (Section 11), provided this store already holds a registered signing key for whichever one signs.
Threshold reconstruction	$M.S.2$ is a leaf: it contributes its own share toward reconstructing its <i>parent</i> $M.S$ ’s secret, an M -of- N threshold of $M.S$ ’s children’s shares — not a signature from $M.S$ or M , and not $M.S.2$ ’s share alone unless $M.S$ ’s own threshold is 1.
Physical custody	If <code>minimum_physical_devices</code> is 2, the shares actually consumed toward that threshold must sit on at least two distinct device ids — regardless of which labels those devices happen to hold.

None of these four answers can be derived from another. That independence is the extension KeyQuorum makes to a plain hierarchical-principal model: instead of one tree carrying one kind of authority (as a CA chain or a HIBE key-derivation tree does), the same tree of dotted labels is the shared index for four composed access structures, each satisfied on its own terms, each owned by a different piece of the system, and each capable of tightening independently — raising a threshold, requiring more physical devices, adding a signing-authority ancestor, or reshaping the label tree itself — without weakening the others.

Note

This is also why extending the model with a new relationship should not mean widening `org_update.rs` (the orchestration layer) itself. A new authenticated-update primitive belongs in whichever module owns the table it actually reads or writes — `key_tree.rs` for tree shape, `private_bridge.rs` for the dotted-label hierarchy and bridge rosters, or `keys.rs` for the hardware-key registry — so that the dimension being extended stays legible, and the other three stay exactly as strict as they were before.

7 Quorum-protected files

`access quorum` protects a file against a split tree the same way `split` escrows a standalone secret — `-leaf` builds the tree in the same lock command.

```
keyquorum split --label "escrow demo" --threshold 2 \
  --leaf alice=alice.pub --leaf bob=bob.pub --generate-keys --register
keyquorum tree <key-id>
keyquorum reconstruct <key-id> --share-file alice.key --share-file bob.pub

keyquorum access quorum --state 0 --source ./secret.txt \
  --encrypted-path ./secret.txt.kqenc \
  --leaf alice=alice.pub --leaf bob=bob.pub --register
keyquorum access quorum --status --id <file-id>
keyquorum access quorum --state 1 --id <file-id> \
  --share-file alice.key --share-file bob.pub
```

<key-id> and <file-id> are printed by `split` / `access quorum -state 0` (“Split key 1” / “Locked file 1”). `-share-file` takes the same key files `generate` wrote (alice.pub / alice.key) or another standard public-key file (PEM, OpenSSH .pub). The private key unwraps every leaf sealed to that hardware key. `tree` still prints leaf node ids for inspection.

7.1 Expiry

`-expires "yyyy-mm-dd hh:mm"` (UTC) gives the locked file a cutoff: the first unlock attempt after that instant deletes the ciphertext and the database row instead of unlocking it.

```
keyquorum access quorum --state 0 --source ./secret.txt \
  --encrypted-path ./secret.txt.kqenc --leaf alice=alice.pub \
  --leaf bob=bob.pub --expires "2026-12-31 23:59"
```

7.2 Verbose unlock

`-state 1 -verbose` prints which shares unwrapped, which subset met the threshold, the physical devices counted toward `minimum_physical_devices`, and any parent approval that was checked. This is the most direct way to confirm *why* an unlock succeeded or failed on a given set of shares.

8 Password-protected files and credentials

Not every secret needs a hardware quorum. `access password` protects a file with a passphrase alone, and `vault` stores ordinary credentials (a username plus a secret) the same way.

```
keyquorum access password --state 0 --source ./secret.txt \
  --encrypted-path ./secret.txt.kqenc
keyquorum access password --state 0 --source ./secret.txt \
  --encrypted-path ./secret-expiring.txt.kqenc --expires "2026-12-31 23:59"
keyquorum access password --state 1 --id 1 --output ./recovered.txt
```

```
keyquorum vault add "Email" --username alice
keyquorum vault get 1
```

Either can also take `-pin` to require a 4-digit PIN, attempt-limited in the style of a FIDO2 PIN, alongside the password. A successful one-time PIN check is cached for one hour; end that window early with, for example:

```
keyquorum pin relock --resource credential --id 1
```

Note

Password-protected files share the same UTC `-expires` cutoff and destructive purge-on-first-touch behavior as quorum-protected files (Section 7).

9 Signatures, export, and sharing

```
keyquorum verify --public-key-file signer.pub --message-file msg.txt \
  --signature-file msg.sig
keyquorum sign --bridge-uid <uid> --node M.A.2 \
  --signing-key-file ma2.sign.key --share-file ma2.key \
  --message-file msg.txt --signature-out msg.kqbs
keyquorum verify --bridge-uid <uid> --as-node M.S.2 \
  --message-file msg.txt --signature-file msg.kqbs

keyquorum export credential 1 --recipient-key-file bob.pub --output cred.kqxb
keyquorum export file 1 --recipient-key-file bob.pub --output file.kqxb
keyquorum export tracked-file report.kqtf --recipient-key-file bob.pub \
  --output report.kqxb

keyquorum share create-file 1 --ttl-seconds 3600 --pin
keyquorum share create-file 1 --expires "2026-12-31 23:59"
keyquorum share redeem-file
```

`verify` works two ways: against a plain public key (`-public-key-file`) for an ordinary Ed25519 signature, or against a private sign bridge's roster (`-bridge-uid` / `-as-node`) for a `.kqbs` artifact — see Section 10.

`export` produces a portable KQXB bundle sealed to a recipient's public key, for a credential, a locked file, or a complete, structurally verified tracked file. Bundles use the same sealed envelope framing as every other letter in this manual; only the recipient's private key can open one. A tracked-file bundle is checked for structure (hashes, parent links and the history chain), not for signatures or trust: it carries every revision the container holds, pending ones included, and the recipient's store judges them.

`-expires` is UTC (yyyy-mm-dd hh:mm). After that instant, redeeming a share link, unlocking, or a routine scan deletes the ciphertext from disk and drops the file's database row. `-ttl-seconds` is a separate, relative share-link lifetime and does not by itself remove the underlying file.

Note

`relay push -expires "2026-12-31 23:59"` stamps the same UTC cutoff on each uploaded envelope. An inbox pull skips expired rows and removes them from the mailbox, so the recipient cannot fetch the file after the date.

10 Private sign bridges

A private sign bridge is an N -person group that can co-sign files. Each node is its own storage: `M.A.1` is an employee, `M.A` is the accounting manager, `M` is a cross-department manager (a CXO). A bridge of `M.S.2`, `M.S.3`, and `M.A.2` therefore notifies **five** stores — those three employees plus department managers `M.S` and `M.A`. The CXO is not in that set unless a member is itself a department manager.

Members receive a sealed copy of a shared Ed25519 key (plus salts). Managers receive roster metadata only, so they can track the live standard. This database never keeps another person's sealed secret; `create` and `remove-member`

write one `.kqpb envelope` per store.

Note

`create` and `remove-member` use the same plan-then-commit pattern: they generate every envelope first, write the files, and only then persist the new (or rotated) bridge in this database. Either both land or neither does — a failed write leaves no live row for that change, and a failed commit deletes the envelopes it just wrote, since they would describe a bridge this store never recorded, and `.kqpb` files are never overwritten (leftovers would block the retry). Either way, simply run the command again. There is no separate “redeliver initial packages” command because a failed create never commits.

Each `-member` must already have a registered **signing** public key under that label (`generate -type signing -label M.S.2 -register`). The roster stores that key. `verify` checks the artifact against the roster-bound personal key, not a public key declared only on the signature.

10.1 Envelope shape

Each `.kqpb` is a cryptographic envelope (the same idea as a KQXB export bundle): the outside names the recipient’s X25519 public key; the inside is `crypto_box`-sealed so only that store’s encryption private key can open it. A mailbox, USB drop, or email can carry the envelope without being able to read the letter. (`.kqbn` eviction notices are the exception — they are public routing slips, not sealed envelopes.)

outside (anyone can see)	inside (recipient only)
KQPB magic, kind	wrap_salt bridge secret (members)
recipient encryption pub	or roster + salts (managers)
	roster includes each member’s personal signing pub
	+ rotate/destroy auth signature

Five people in the example above means five envelopes, each addressed to a different public key. Operators can copy those files out of band, or push them through the mailbox relay (`keyquorum relay push`) so each store can `relay pull -import` locally.

```
# Each member needs a registered signing public key under their label:
keyquorum generate --type signing --label M.S.2 --register \
  --public-key-out M.S.2.sign.pub > M.S.2.sign.key
# ...same for M.S.3 and M.A.2

# On any machine that can see the tree (and manager encryption pubs):
keyquorum bridge private create 1 \
  --member M.S.2 --member M.S.3 --member M.A.2 \
  --supervisor M.S=SoftwareManager.pub --supervisor M.A=AccountingManager.pub \
  --self M.S.2 --output-dir ./bridge-packages --label eng-acct

# Each other person imports into *their* database:
keyquorum --db ma2.sqlite bridge private import \
  --file ./bridge-packages/M.A.2.kqpb --share-file Accounting2.key

# Sign with the bridge key + the member’s personal signing key:
keyquorum sign --bridge-uid <uid> --node M.A.2 \
  --signing-key-file Accounting2.sign.key --share-file Accounting2.key \
  --message-file report.pdf --signature-out report.kqbs

# Peer verifies with their membership and the artifact:
keyquorum --db ms2.sqlite verify --bridge-uid <uid> --as-node M.S.2 \
  --message-file report.pdf --signature-file report.kqbs
```

10.2 Revocation and rotation

If M.S revokes M.S.3's hardware key, that label is dropped from every live private bridge. Ordinary **revoke** (not only **-evict**) prints the notify list and writes a **.kqbn** notice **after** the hardware revoke commits. Remaining members must rotate — the departed person still holds the old bridge secret. A remaining member then runs:

```
keyquorum bridge private remove-member <uid> --member M.S.3 \
  --node M.S.2 --share-file Software2.key --output-dir ./bridge-packages-rotate
# Copy the new packages to M.S.2, M.A.2, M.S, M.A, and M.S.3
# (or 'keyquorum relay push --dir ./bridge-packages-rotate'). Package
# writers name files by recipient and never overwrite, so reuse the
# original ./bridge-packages here only after clearing it out.
```

A two-person bridge is destroyed when one member is removed.

10.3 Evicting a hardware key

Banning a hardware key is **revoke <hardware-id>**. That always drops pairings and whitelist rows for every live leaf sealed to that token, and drops that label from every live private sign bridge. **-evict** additionally PSS-refreshes survivors of that leaf (**-key-id** / **-node**, or the unique leaf the token backs). Eviction needs a parent threshold of at least 2, and every remaining active sibling must be a hardware-backed leaf — a 1-of-*N* parent is refused. Binds between survivors stay. **-share-file** is that sibling's actual key file (**.pub** / **.key** from **generate** or **split-generate-keys**, or a PEM / OpenSSH public key). A **.pub** file uses a sibling **.key** when present.

```
keyquorum split --label "team escrow" --threshold 2 \
  --leaf alice=alice.pub --leaf bob=bob.pub --leaf carol=carol.pub \
  --generate-keys --register
keyquorum tree <key-id>
keyquorum revoke <carol-hardware-id> --evict \
  --share-file alice.key --share-file bob.pub
# pin a leaf when the token backs more than one:
#   --key-id <key-id> --node carol
```

11 Authenticated updates: reissue and restructure

Replacing someone's token, or changing the shape of the split tree, has to reach every store that held the old picture. Both changes ship as the same sealed **.kqpb** envelope a private bridge uses, so the mailbox routes them without being able to read them — but the letter inside is *signed* by an authorizing label, so each store decides for itself whether to apply it.

11.1 The four-part authorization rule

A store applies an update only when all four of these hold:

- **Addressed here** — the envelope's recipient key is an unrevoked encryption key this store has registered under the label named inside the letter. Re-addressing a letter to another mailbox gets it rejected.
- **Authorized** — the signer is the subject itself or one of its ancestors in the dotted hierarchy (M over M.S over M.S.2), and this store already holds a registered signing key for that label. A peer branch cannot restructure your tree or swap your token.
- **Signed** — Ed25519 over a domain-separated hash that covers every field, including the recipient label and its public key.
- **In order** — a reissue must be exactly one past the last one this store applied for that person; a restructure must carry a public generation strictly greater than the one stored. Replays, stale envelopes, and skipped sequences change nothing.

```
# The authorizing label needs a registered signing key:
keyquorum generate --type signing --label M --register \
  --public-key-out M.sign.pub > M.sign.key
```

```

# Restructure: after 'add', 'revoke --evict', or 'bind', announce the new
# shape. Each active leaf gets its own visible slice at the next generation.
# Root (M) is effective immediately. A label with a parent, such as M.S,
# writes a proposal and waits. Each write below gets its own --output-dir
# and its own push: these can overlap in which leaves they address (a
# proposal for M.S can name the same leaves the root's own restructure
# just did), and package writers never overwrite an existing file.
keyquorum tree restructure 1 --as M --signing-key-file M.sign.key \
  --output-dir ./updates-root
keyquorum relay push --dir ./updates-root

keyquorum tree restructure 1 --as M.S --signing-key-file M.S.sign.key \
  --output-dir ./updates-ms-proposal
keyquorum relay push --dir ./updates-ms-proposal

keyquorum tree countersign 1 --as M --signing-key-file M.sign.key \
  --output-dir ./updates-countersign
keyquorum relay push --dir ./updates-countersign
# the same countersignature from a slot:
# keyquorum tree countersign 1 --as M --device ./usb --slot M \
#   --output-dir ./updates-countersign

# Reissue: M.S.2 lost their token and generated a replacement.
keyquorum reissue --node M.S.2 --key-id 1 \
  --encryption-public-key-file M.S.2.new.pub \
  --as M --signing-key-file M.sign.key --revoke-previous \
  --output-dir ./updates-reissue
keyquorum relay push --dir ./updates-reissue

```

Each store applies these the same way it applies a bridge envelope — the kind byte in the header decides which it is:

```

keyquorum --db M.S.1.sqlite relay pull --import --share-file M.S.1.key
keyquorum --db M.S.1.sqlite bridge private import --file M.S.1.kqpb \
  --share-file M.S.1.key
keyquorum --db M.S.1.sqlite updates # what this store has applied

```

11.2 What a reissue changes

A reissue reaches every store whose own slice named that person, plus every party of every live private bridge they are on — nobody else, since nobody else held the key. It repoints that person's tree leaves and bridge roster entries, and drops the leaf's sealed share, which was wrapped to the retired token and cannot be opened by its replacement. Recover it from the quorum, or reseal with `bind -public-key-file`.

Watch out

The shared secret of a private bridge is *not* rotated by a reissue. Run `bridge private remove-member` to roll that generation if the retired token could have been compromised.

The subject's own copy is addressed to the **incoming** encryption key, so their replacement device must have registered it (`generate -register`, or `register`) before importing — which is also what lets it open the letter.

11.3 Restructure vs. publish

`tree restructure` differs from `tree publish` in exactly this way: `publish` replaces a relay document with a store's own view, while a restructure hands each person a slice they can verify the authority for.

A restructure whose authorizer is the root is applied as soon as a store accepts the envelope. A restructure whose authorizer has a parent (M.S under M) is a proposal until that parent runs `tree countersign`; import accepts that change only with the parent's countersignature. A routine employee reissue by that employee's own parent (M.S reissuing M.S.1) stays a single signature.

12 Moving a key between devices

`keyquorum transfer` copies or moves an active identity between two devices that are both open. Each side has its own container and its own database; the global `-db` is not used. Enroll a slot before the first transfer so this device has an active identity for that label.

```
keyquorum transfer enroll --device ./usb --db ./alice.sqlite --label M.S
keyquorum transfer copy \
  --from-device ./usb --from-db ./alice.sqlite \
  --to-device ./usb2 --to-db ./bob.sqlite \
  --label M.S
keyquorum transfer move \
  --from-device ./usb --from-db ./alice.sqlite \
  --to-device ./usb2 --to-db ./bob.sqlite \
  --label M.S --descendants key-only
keyquorum transfer list --db ./bob.sqlite
keyquorum transfer list --db ./alice.sqlite --all
keyquorum transfer recover \
  --from-device ./usb --from-db ./alice.sqlite \
  --to-device ./usb2 --to-db ./bob.sqlite \
  --transaction <hex-id>
```

The identity `id` is stable; the dotted label is only the current place in the hierarchy. Possession on a device is **active** (the slot secret is here), **ghost** (the hierarchy row remains and the secret does not), or **absent**. `transfer list` hides ghosts unless `-all` is set.

- **COPY** leaves the source active and records that the destination may hold a legitimate copy.
- **MOVE** leaves the source row active until the destination has committed and the source slot token is gone. The ghost row is written only after that deletion, so a reported ghost cannot still be opened by `keyquorum-device`. A transfer that stops before the destination commits keeps the usable copy on the source.

`transfer recover` finishes a transfer whose destination already committed, or rolls back one that did not.

`-descendants` selects what travels with the label: **key-only** (the default), **direct-children**, or **all-descendants** (the full subtree). A parent can move without its children, and a child can move while the parent stays. `-as` names the active identity authorizing the transfer and defaults to `-label`.

Note

A ghost keeps hierarchy and provenance and holds no usable private key. It cannot sign, satisfy a quorum, authorize an import, or be exported. An active child under a ghost ancestor stays usable. Ghosts do not authorize an incoming key.

An empty receiver accepts the package. A receiver that already holds active identities accepts an incoming key only when it is a descendant of one of those identities. The same identity `id` with the same public keys reconciles; the same `id` or the same label with different key material is refused.

12.1 Package format and audit

The package magic is `KQTX`. The source device key signs it, and it is bound to the destination device `id`. Replay, tamper, and a bad source are refused. Every attempt is audited without private key material; SQLite stores the transfer state and the audit row. The raw `KQTX` package stays off the mailbox.

12.2 Transferring through the mailbox

When the two devices are not open together, the commands below seal that package, or a slot relocate, into a `.kqpb` letter, and the mailbox stores only that letter.

```
# Bob prepares the receiving side first: a slot to collect into, and a
# published descriptor so relay-finalize can later fetch his verify key.
# 'provision' prints the new slot's encryption and signing public keys;
```

```

# save the printed encryption hex to bob-slot.pub and send it to Alice
# out of band (relay-send needs it before either device is online together).
keyquorum device provision ./usb2 --label recv
keyquorum device publish ./usb2 --url https://relay.example.com

keyquorum device publish ./usb --url https://relay.example.com
keyquorum transfer relay-send \
  --operation move \
  --from-device ./usb --from-db ./alice.sqlite \
  --label M.S \
  --to-device-id <hex-device-id> \
  --recipient-key-file ./bob-slot.pub \
  --url https://relay.example.com
keyquorum transfer relay-collect \
  --to-device ./usb2 --to-db ./bob.sqlite \
  --slot recv \
  --from-device-id <hex-source-id> \
  --url https://relay.example.com
keyquorum transfer relay-finalize \
  --from-device ./usb --from-db ./alice.sqlite \
  --slot M.S \
  --to-device-id <hex-device-id> \
  --url https://relay.example.com
keyquorum device relay-relocate \
  --from ./usb --label M.S \
  --to-device-id <hex-device-id> \
  --recipient-key-file ./bob-slot.pub \
  --url https://relay.example.com
keyquorum device relay-accept ./usb2 --slot recv \
  --url https://relay.example.com
keyquorum device relay-drop ./usb --label M.S \
  --to-device-id <hex-device-id> \
  --relocate-id <hex-relocate-id> \
  --url https://relay.example.com

```

relay-send leaves the source prepared. **COPY** stays active. **MOVE** deletes the source slot token and writes the ghost only when **relay-finalize** checks the destination's signed acknowledgement. **relay-relocate** prints a relocate id and leaves the source slot in place until **relay-drop** checks an acknowledgement signed for that id — an acknowledgement from an earlier relocation of the same slot does not match. **relay-accept** acknowledges a slot that is already installed with the same keys again, so a retry after a failed upload finishes.

The mailbox keeps device letters and acknowledgements for 30 days and never deletes them on acknowledgement. Load a **device.push** key for sends and publishes, and a **device.pull** key bound to the recipient fingerprint for collects, finalizes, and drops. **relay-collect** and **relay-accept** also need **device.push** to post the acknowledgement (**-push-key**, or a stored key of that scope).

13 Sending a file

deliver seals one file to another label. You sign it with your signing key and seal it to the encryption key your store has registered for the recipient. The recipient opens it: that step verifies your signature against the signing key their store holds for you. They then answer with a signed accept or reject, sealed back to you. Letters and answers are ordinary **.kqpb** envelopes that **relay push** / **relay pull** carry.

```

keyquorum deliver send --file report.pdf --to M.A --as M.S.1 \
  --slot ./usb=M.S.1 --push
keyquorum --db david.sqlite relay pull --output-dir ./mail
keyquorum --db david.sqlite deliver open --file ./mail/7.kqpb \
  --slot ./usb=M.A --save-dir ./received --push-ack

# The acknowledgement is sealed back to the sender -- pull it into the

```

```
# sender's own store (a different mailbox than David's ./mail above)
# before answering it.
keyquorum relay pull --output-dir ./mail
keyquorum deliver ack --file ./mail/8.kqpb --slot ./usb=M.S.1
```

`-output-dir` / `-ack-dir` write the envelope to disk instead of (or as well as) `-push` / `-push-ack`. `-reject` refuses the file and says so in the answer.

To send a file whose revisions and signatures should travel with it, and where an unsigned newer edit must never go out, track it instead and use `file share` (section 14).

14 Tracked files and history

A *tracked file* carries its own provenance. Where `deliver` sends a native file once, `keyquorum file` keeps a `.kqtf` container that holds every revision of the file, the signatures over them, the rules they are judged by, and an append-only, hash-chained history of everything that happened to it. The container is the authority: SQLite only keeps a cache of it, and a recipient can verify a tracked file without the sender's database.

14.1 What a tracked file holds

- **A stable identity.** A random 128-bit `file_id` is allocated once, when the file is first tracked, and never changes on rename, move, device transfer or new revisions. The logical file name is display metadata, not identity.
- **A revision graph.** Each revision records its parent revisions (none for the first, one for an edit, two for a merge), a content commitment bound to the file (so it is not a global fingerprint of the plaintext), the author's label and identity, a UTC time, the topology generation, and the hash of the file's policy. Its 32-byte `revision_id` is a domain-separated hash of all of that, so a revision cannot be re-parented or re-attributed without changing its id. Every revision also has a generated label, `R<file-name>-<UTC time>-<HCP label>`, and may have a user label; labels are for people, the id is what is signed.
- **Proofs.** Content signatures by a revision's author, countersignatures by an approver, and finalizations by the scope owner or an ancestor. They are ordinary Ed25519 signatures over domain-separated preimages (`KQ-FILE-REVISION-v1`, `KQ-FILE-COUNTERSIGN-v1` and `KQ-FILE-FINALIZE-v1`), made and checked by the same signing code as the rest of KeyQuorum.
- **A policy.** The HCP scope the file belongs to and what each kind of author needs before a revision is trusted (below).
- **A history.** Sequenced events, each linked to the hash of the one before; the last hash is the file's *history root*. Changing, removing or reordering any earlier event breaks the chain. Events record outcomes and labels, never passwords, PINs, shares, bearer tokens or private keys. Every detail an event may carry comes from a fixed list of safe keys; anything else is refused before it is written. Some events are also signed by the person who caused them (below).

The container format is KQTF, currently version 7. Version 6 added finalization proofs and version 7 optional event signatures; each is written only when a file has one, so most files stay at an older version, and versions 4 to 6 still open. It is not a sealed envelope and does not wrap or modify the native file format: `file checkout` writes a revision's bytes back out as an ordinary file.

When tracking starts, and what stays the same

Signing a native file that has no container yet is the first content signature: `file sign report.txt -as M.A -slot ...` allocates the file id, records revision R1, signs it and writes `report.txt.kqtf`, the same steps as `file track`. The file's scope is the signer's own label unless `-scope` names another. Files nobody signs this way stay untracked. Tracking begins at the first *trusted* signature: if the first revision would still wait for a countersignature (a descendant signing alone), nothing is written and the command says who can start tracking; the explicit `file track` may still begin pending. A second `file sign` on the same native path is refused instead of minting a second identity. The check is by path: the same bytes signed from a copy at another path start a separate tracked file, because a file's identity is a random id and not a fingerprint of its content.

The file's identity does not depend on its name, path or holder. `file rename` (scope owner or an ancestor, proven

with `-slot`) changes only the name shown, appends a `FILE_RENAMED` event with the old and new names, and leaves the file id and every revision id alone. Copying or moving the `.kqtf` file changes nothing either. `file checkout` writes a revision's bytes back out exactly, whatever the format.

`file status` also prints two ids that can differ, because a newer edit is not a newer trusted version (and says when a revision is final):

- `current_revision_id`: the sole head, or none when the history has forked.
- `trusted_revision_id`: the latest revision this store trusts.

Both are worked out on demand from the store's keys and are never stored in the container, which carries no trust state. They are views relative to the store that shows them, not claims the container makes, so a copy of the file judged with other keys can show a different trusted revision. On a fork there is no single current revision, and the trusted one is only the most recently stored trusted revision across both branches: a display value, never a branch winner and never the revision a share would send.

14.2 Trust: who may change a file

A revision is *trusted* only when its author's content signature verifies against the signing key your store has registered for that label *and* the file's policy is met. The default policy (`file track`) is, relative to the file's scope:

Author	Needs
The scope owner	Their own signature.
A descendant (e.g. M.A.1 in scope M.A)	Their own signature and a countersignature by their <i>direct parent</i> (M.A).
An ancestor	Their own signature.
Cross-branch	Their own signature and either a private bridge's approval of that revision or a countersignature by the scope owner.
Unrelated	May not create trusted revisions.

A revision is therefore *pending* until its signatures are complete, and *denied* if a signature is present but wrong. A newer revision never becomes trusted just because it is newer, and time stamps never decide anything.

Each revision also records the topology generation (the version of the published key tree) it was made under. Your store judges it only under a generation it has held; a revision from a generation your store never saw is left *pending* (`MissingTopologyEvidence`) rather than judged against today's tree. Generation 0 means no published tree.

The rules are chosen when the file is tracked and are fixed for its life. Each role has its own option, which takes `forbidden`, `author` or `author+parent`:

- `-owner-rule`
- `-descendants-rule`
- `-ancestors-rule`
- `-cross-branch-rule`, which may also take `author+bridge-or-owner` or the stricter `author+bridge+owner`.

The defaults are the table above. A rule that cannot be met — a forbidden owner, a bridge for anyone but a cross-branch author, a parent countersignature where there is no parent — is refused before anything is written. `file policy` prints the rules and the `policy_hash` that every revision carries, so later verification knows which rules applied. When a countersignature arrives, the history records its `relationship` to the author (for example `DIRECT_PARENT`) and the policy decision records its `reason`.

Revision labels are `R<name>-<UTC time>-<HCP label>` with the time to the millisecond where the clock has it, so adjacent check-ins get different labels. A label is part of the immutable revision: changing it would mean a different revision, never an edit of the existing one.

```
keyquorum file track report.txt --scope M.A --as M.A --slot ./usb=M.A
keyquorum file checkin report.txt.kqtf --from edited.txt --as M.A.1 --unsigned
keyquorum file sign report.txt.kqtf --as M.A.1 --slot ./usb=M.A.1
keyquorum file countersign report.txt.kqtf --as M.A --slot ./usb=M.A
keyquorum file status report.txt.kqtf
```

Bridge approval

A cross-branch author can be approved through a private bridge instead of by the scope owner. A member of a live private bridge whose members reach from the author to the file's scope signs an approval of that exact revision:

```
keyquorum file bridge-approve report.txt.kqtf --bridge <uid> --as M.S.1 \
--slot ./usb=M.S.1
```

The approval is kept only in the approving store and never written into the `.kqtf`, so the file never names the bridge or its members. Each time a revision is judged it is checked again: the bridge must still be live at the same generation and the signer still a member. A bridge's supervisors do not count, and a bridge merely existing between two labels approves nothing. A recipient whose store holds no such approval sees the revision as pending. Under `author+bridge-or-owner`, a valid scope-owner countersignature then substitutes for the bridge approval. A trusted cross-branch policy decision records the alternative that satisfied it as `satisfied_by=BRIDGE`, `SCOPE_OWNER`, or `BRIDGE_AND_SCOPE_OWNER`. It never records a bridge id, generation, signer, member, or roster, but `BRIDGE` and `BRIDGE_AND_SCOPE_OWNER` do reveal that a bridge approval took part in the decision, and that stays in the `.kqtf` and in any export of it.

With the stricter `author+bridge+owner` rule, the scope owner's countersignature does not replace the bridge approval. The receiving store must hold and validate that revision-specific bridge approval *and* the container must carry the scope owner's valid countersignature before the revision is trusted.

Finalization

Trusted is not final. The scope owner or one of its ancestors can mark a trusted revision final:

```
keyquorum file finalize report.txt.kqtf --as M.A --slot ./usb=M.A
```

This adds a signed finalization proof and a `REVISION_FINALIZED` event. Like trust, finality is worked out by each store from its own keys; a store without the finalizer's key does not see the revision as final. A merge of two final revisions is a new revision and starts neither trusted nor final.

Security

Your store supplies only public signing keys, and it accepts a key only for the identity it knows for that label (the one enrolled by `transfer enroll`, or else one derived from the label). A container cannot bring its own keys. A ghost identity (after a `transfer move`) cannot author or sign anything new; what it signed earlier still verifies.

14.3 Sharing falls back to the last trusted revision

`file share` sends the head (or the revision named with `-revision`) when it is *trusted*. If it is not — say someone checked in an edit without signing it — the letter carries its nearest trusted *ancestor* instead. Trusted work on another branch is never substituted, and the unsigned revision never leaves your copy: the container in the letter holds only the delivered revision and its ancestors. If no ancestor is trusted, nothing is sent and the refusal is recorded. The refusal says why: *denied by policy* when the file's rules refuse the revision outright (an unrelated or forbidden author, evidence that does not verify), and *no trusted revision* when it is only waiting for a signature.

```
keyquorum file share report.txt.kqtf --to M.B --as M.A --slot ./usb=M.A \
--output-dir ./outbox
keyquorum --db mb.sqlite file receive --letter ./outbox/<id>.kqpb \
--slot ./usb=M.B --ack-dir ./acks --out report.txt.kqtf
keyquorum file ack report.txt.kqtf --ack ./acks/<id>-ack.kqpb --slot ./usb=M.A
```

Letters are KQPB envelopes of their own kind, carried by the relay like any other letter. The sender's signature on the letter is *transport* authentication only: the recipient still judges the delivered revision by the file's own policy against their own store, and refuses it if it is not trusted there. The recipient's store must also hold the encryption key that opened the letter under the recipient label the letter names, so a letter sealed to one person cannot credit the delivery to another; `deliver open` makes the same check. The signed letter also names every proof it carries for the delivered revision, and `file receive` refuses a letter whose container does not match. A rename made before sharing travels with the letter, and so does any later history that does not concern a revision outside the delivered

one. `-into` merges a letter into a copy you already hold. Either way the recipient seals back a signed accept or reject, and `file ack` records it once in the sender's history. Receiving the same letter again records nothing new; it only seals the answer again.

`file receive` also says how the delivered history compares with what your store accepted before for that file: **FIRST**, **REPLAYED** (the same revision and history again), **NEWER** (its history passes through every history root you accepted before, holds those revisions and adds one) or **NOT_NEWER** (an older copy, another branch, or the same revisions on a rewritten history). It is recorded on the delivery. **NOT_NEWER** is a warning, not a refusal: a trusted older revision is still accepted.

`file send-history` seals a file's event history alone (a KQHS snapshot, no content) to another label, with its root and event count signed by you. `file open-history` checks the signature and the snapshot and, with `-against`, compares it with your copy: **SAME**, **LOCAL_AHEAD**, **REMOTE_AHEAD** or **DIVERGED**. It is not answered.

```
keyquorum file send-history report.txt.kqtf --to M.B --as M.A \
--slot ./usb=M.A --output-dir ./outbox
keyquorum --db mb.sqlite file open-history --letter ./outbox/<id>.kqpb \
--slot ./usb=M.B --against report.txt.kqtf
```

14.4 Sending and receiving a tracked file from the command line

A tracked file travels as a *letter*: a sealed `.kqpb` file that holds no secret. It is signed by the sender and sealed to the recipient's public key, so it can cross any channel. The commands that make and open a letter are the same whether or not a relay is used; only the way the letter and the answer get from one person to the other differs. Each person needs only their *own* drive, plugged into their *own* computer, and only at the moment they sign or open something. Nobody hands over a key drive.

The examples send `report.txt.kqtf` from M.A to M.B. Each command names the person who runs it and the computer it runs on.

Step 0, once, before either way: exchange public keys

Each store needs the other's public keys: the sender's signing key (to check the letter) and the recipient's encryption key (to seal it). They are public, so any channel will do.

```
# 0a. on M.A's computer (M.A's drive in): print M.A's public keys, send them to M.B
keyquorum-device public ./usb --label M.A
# 0b. on M.B's computer: register what M.A sent
keyquorum --db mb.sqlite register --type encryption --label M.A \
--public-key-file a-enc.pub
keyquorum --db mb.sqlite register --type signing --label M.A \
--public-key-file a-sign.pub
# 0c. M.B does the same in the other direction: prints their keys with
# keyquorum-device public, M.A registers them in ma.sqlite
```

When the API is not available: you carry the files

No URL and no API key. The letter and the answer are plain files; move them by email, a shared folder, or a USB stick that is only a courier.

```
# 1. M.A's computer, M.A's drive in: seal the letter into a folder
keyquorum --db ma.sqlite file share report.txt.kqtf --to M.B --as M.A \
--slot ./usb=M.A --output-dir ./outbox

# 2. Carry ./outbox/<id>.kqpb to M.B's computer (any channel)

# 3. M.B's computer, M.B's drive in: open the letter and seal the answer
keyquorum --db mb.sqlite file receive --letter ./<id>.kqpb \
--slot ./usb=M.B --ack-dir ./acks --out report.txt.kqtf
# --into report.txt.kqtf merge into a copy M.B already holds (instead of --out)
```

```
# --reject                refuse the letter (instead of --out or --into)

# 4. Carry ./acks/<id>-ack.kqpb back to M.A's computer (any channel)

# 5. M.A's computer, M.A's drive in: record the answer
keyquorum --db ma.sqlite file ack report.txt.kqtf --ack ./<id>-ack.kqpb \
--slot ./usb=M.A
```

To send M.B's changes back, swap the roles and repeat steps 1 to 5. If both copies have moved on, the receiver ends up with two heads:

```
# 6. on whoever received: join the heads, then settle any conflict
keyquorum --db ma.sqlite file merge report.txt.kqtf --as M.A
keyquorum --db ma.sqlite file review report.txt.kqtf
keyquorum --db ma.sqlite file resolve report.txt.kqtf --keep right \
--as M.A --slot ./usb=M.A
keyquorum --db ma.sqlite file sign report.txt.kqtf --as M.A --slot ./usb=M.A
```

A no-drive alternative to steps 1 to 5 is to carry the whole .kqtf and run `file import report.txt.kqtf -from theirs.kqtf -as M.B`. It skips the signature and trust check that a letter carries.

When the API is available: the relay carries the files

A relay is a mailbox that stores sealed letters and cannot open them. It adds no security the letter lacks. It saves the two people from handing the files over and lets them act at different times. You need the relay's URL and API keys from its operator. Each key has one scope: an *inbox push* key sends a letter or an answer, an *inbox pull* key downloads what is waiting, so each person usually holds both.

```
# 1. Once per person, on their own computer: store the relay URL and key
# (the key is prompted for when it is not on the command line)
keyquorum --db ma.sqlite loadkey --url https://relay.example

# 2. M.A's computer, M.A's drive in: seal the letter and upload it
keyquorum --db ma.sqlite file share report.txt.kqtf --to M.B --as M.A \
--slot ./usb=M.A --push

# 3. Later, M.B's computer: download what is waiting for M.B
keyquorum --db mb.sqlite relay pull --output-dir ./inbox

# 4. M.B's computer, M.B's drive in: open the letter, upload the answer
keyquorum --db mb.sqlite file receive --letter ./inbox/<n>.kqpb \
--slot ./usb=M.B --push-ack --out report.txt.kqtf

# 5. Later, M.A's computer: download the answer
keyquorum --db ma.sqlite relay pull --output-dir ./answers

# 6. M.A's computer, M.A's drive in: record the answer
keyquorum --db ma.sqlite file ack report.txt.kqtf --ack ./answers/<n>.kqpb \
--slot ./usb=M.A
```

Sending changes back and settling conflicts work exactly as in the previous case (swap the roles, then the merge, review, resolve and sign commands above). `-url` and `-api-key` can be given on `share` and `receive` instead of using the stored key. These environment variables work too:

- KEYQUORUM_RELAY_URL
- KEYQUORUM_RELAY_API_KEY

Downloaded letters are named by the relay's own number (<n>.kqpb). The official client only talks to a relay that proves it holds a KeyQuorum-signed certificate. Everything else, including the sender's signature, the recipient-key check, the history comparison and the signed answer, is the same as without the relay.

Watch out

Without the relay a drive is needed only for the person who is signing or opening at that moment. The only cases that need two devices open together are moving or copying an identity between devices (**keyquorum transfer copy|move**), and the browser lab, which simulates everyone on one machine. A merge proposal, a file request and a change request all travel as letters the same way (see the two sections below).

14.5 Copies, forks and merging

Two holders of a file can each extend it. **file import** brings another copy's revisions and proofs into yours: nothing is overwritten, and if both sides added revisions the file now has two heads. **file merge** then tries a conservative automatic merge of UTF-8 text, line by line, against the heads' nearest common ancestor:

Outcome	Meaning
FastForward	One head already contains the other.
AlreadyEquivalent	Both heads hold the same content.
CleanMerge	The edits do not touch the same lines; a new two-parent revision is recorded.
RequiresHuman	The edits overlap differently, or there is no single common ancestor.
PolicyBlocked	The file's policy turns automatic merging off (track -no-auto-merge).
UnsupportedContent	The content is not UTF-8 text.

A merge revision never inherits its parents' signatures: it starts *pending* and has to be signed (and countersigned where the policy says so) like any other revision. When a person has to decide, **file merge** records the conflict and names a reviewer: the most recent neutral prior owner of the file, else the most senior eligible owner, else (for a tie across sectors) the larger sector, else the nearest common senior ancestor. Only owners whose own revisions are trusted qualify, the authors of the conflicting edits never review their own collision, and if nobody qualifies the conflict is recorded as unresolved rather than guessed; then the scope owner or one of its ancestors decides.

The reviewer, or one of their ancestors, settles the conflict with **file resolve**: keep one side as it is (**-keep left** or **-keep right**) or supply an edited result (**-from**). The result is a new two-parent revision signed as the reviewer's own content and recorded as **CONFLICT_RESOLVED**; the file's rules then judge it like any other revision. **-reject** instead refuses an untrusted proposed merge at the head (**MERGE_REJECTED**): the proposal stays in the history, **file sign** refuses it, and the resolution builds on it. A conflicting author can never decide.

```
keyquorum file resolve report.txt.kqtf --keep left --as M.A --slot ./usb=M.A
keyquorum file resolve report.txt.kqtf --from merged.txt --as M.A \
--slot ./usb=M.A
keyquorum file resolve report.txt.kqtf --reject --as M.A --slot ./usb=M.A
```

```
keyquorum file import report.txt.kqtf --from their-copy.kqtf --as M.A
keyquorum file merge report.txt.kqtf --as M.A
keyquorum file review report.txt.kqtf
keyquorum file diff report.txt.kqtf
keyquorum file graph report.txt.kqtf
```

file review prints each side's changed lines with who wrote each line, the merge outcome and the reviewer. When the only head is a merge that waits to be signed, **file review** shows what each parent changed and what the merge changed (**LEFT PARENT**, **RIGHT PARENT**, **MERGED**) with its trust state, and **file merge** opens the interactive review for it. Revisions are shown by their user label first, when they have one, then the generated label and the short id. A build with **-features tui** adds **review -interactive**: the same view in the terminal, with removed text on a red and added text on a green background, Vim-style keys and mouse hover to see who wrote a line. Press **za**, **zc** or **zo** to fold, close or open the change under the cursor (a folded change shows its first line and a count), and **zM** or **zR** to fold or open every change on the side. Two unchanged lines are shown on each side of a change, dimmed and marked as unchanged, and **c** hides or shows them; a fold hides them with the change, and the printed **file review** lists changed lines only. Press **Space** to pick the change under the cursor on either side, **:compose** to see the result (the common ancestor with the picked changes applied; picks that touch the same lines are refused),

and `:edit` to change that result in `$VISUAL` or `$EDITOR`, in an owner-only temporary file that is removed afterwards. `:as LABEL` and `:slot CONTAINER=LABEL` (or `-as` and `-slot` on review `-interactive`) say who is acting; after that `:accept left`, `:accept right`, `:accept result`, `:reject`, `:sign` and `:finalize` run the real file `resolve`, file `sign` and file `finalize` with the terminal handed back for the passphrase, so the reviewer and scope rules are those commands', and the review then shows the history as it stands. `:accept result` settles the conflict when one waits for its reviewer; on a clean merge, which has no conflict, it checks the result in as a new signed revision (file `checkin`) on top of the merge. Press `/` to type a query and `Enter` to search; `n` and `N` repeat it forward and backward. Matching ignores case, looks for a substring in the active pane, and wraps around. On such a build, file `merge` opens it by itself when a person has to decide and the terminal is interactive. The review decides no trust and edits no history itself: every action above is the real command run as the label and slot you gave, and it is refused when that command would refuse you. Without `:as` and `:slot` the commands only name the CLI command to run.

Watch out

Commands replace a container under an exclusive `<name>.kqtf.lock`, and refuse to write if the container changed since the command read it; run the command again. If a command was killed and left the lock behind, the error names the lock file to remove.

14.6 Terms for proposing a change

Three words describe someone asking for, or offering, a change to a file. Each names something this version does. A request only asks: it delivers nothing and changes nothing, and the answer to it only says yes or no. The file or the change itself always travels the ordinary way, as a letter (file `share`) whose revision is judged by the file's own rules.

Term	Meaning and how it works
Merge proposal	Prepared changes submitted for review: a merge revision, made automatically from two heads or by an author outside the file's scope, that stays Pending until the reviewer, the scope owner or a bridge decides. The reviewer settles it with <code>file resolve</code> or refuses it with <code>-reject</code> . The manual also calls it a proposed merge.
Change request	Asking someone to change a file ("please change this file"), before any change exists: <code>file request -change</code> with a short message. The holder reads it (file <code>open-request</code>) and answers accept or decline (file <code>answer-request</code>).
File request	Asking the holder to send a file, usually one you do not hold: <code>file request</code> . An accepted file request is served with <code>file share</code> .

Use *change request* for "please change this file" and *merge proposal* once the author has already prepared the changes.

14.7 Asking for a file, or for a change

A request is a small signed letter (`.kqpb`) sealed to the holder, and the answer is another, sealed back to the requester. They travel exactly as in the previous section: as plain files you carry, or through the relay. The holder checks the requester's signature against the key their own store holds for that label, and the requester checks the holder's answer the same way. A request never opens as a delivery letter, or the other way round.

What a history records, when a copy of the file is named: the request (`FILE_REQUESTED` or `CHANGE_REQUESTED`) with its id and kind, and the answer (`REQUEST_ANSWERED`) with the decision. The message of a change request is shown to the holder and is *never* recorded. Opening the same request or answer twice records it once, and a request already answered one way cannot be answered the other.

When the API is not available: you carry the files

The public keys of both people are exchanged and registered first (step 0 in the previous section). M.B asks M.A:

```
# 1. M.B's computer, M.B's drive in: ask for a file M.B does not hold yet
keyquorum --db mb.sqlite file request --file-id <32 hex digits> \
--name report.txt --to M.A --as M.B --slot ./usb=M.B --output-dir ./requests
```

```
# or ask for a change to M.B's own copy (message up to 1024 bytes)
keyquorum --db mb.sqlite file request report.txt.kqtf --change \
  --message "the total in row 3 is wrong" --to M.A --as M.B \
  --slot ./usb=M.B --output-dir ./requests

# 2. Carry ./requests/<id>-request.kqpb to M.A (any channel)

# 3. M.A's computer, M.A's drive in: read it; --file records it in M.A's copy
keyquorum --db ma.sqlite file open-request --letter ./<id>-request.kqpb \
  --slot ./usb=M.A --file report.txt.kqtf

# 4. M.A's computer: answer (accept or decline); --file records the answer
keyquorum --db ma.sqlite file answer-request --letter ./<id>-request.kqpb \
  --decision accept --slot ./usb=M.A --file report.txt.kqtf --ack-dir ./answers

# 5. Carry ./answers/<id>-answer.kqpb back to M.B (any channel)

# 6. M.B's computer, M.B's drive in: read the answer (--file records it
# against the request M.B's copy shows it made)
keyquorum --db mb.sqlite file open-answer --answer ./<id>-answer.kqpb \
  --slot ./usb=M.B --file report.txt.kqtf
```

If M.A accepted a *file request*, M.A now sends the file with the `file share` steps of the previous section. If M.A accepted a *change request*, the change reaches M.A as a revision the ordinary way: M.B checks it in on their copy and sends it with `file share`, and it arrives as a merge proposal.

When the API is available: the relay carries the files

Store the relay URL and key once per person (loadkey, as before). Only the commands that move a file change:

```
# 1. M.B: the same file request commands, with --push in place of --output-dir
keyquorum --db mb.sqlite file request --file-id <32 hex digits> \
  --name report.txt --to M.A --as M.B --slot ./usb=M.B --push

# 2. Later, M.A's computer: download what is waiting, then read the request
keyquorum --db ma.sqlite relay pull --output-dir ./inbox
keyquorum --db ma.sqlite file open-request --letter ./inbox/<n>.kqpb \
  --slot ./usb=M.A --file report.txt.kqtf

# 3. M.A answers and uploads the answer
keyquorum --db ma.sqlite file answer-request --letter ./inbox/<n>.kqpb \
  --decision accept --slot ./usb=M.A --file report.txt.kqtf --push-ack

# 4. Later, M.B's computer: download the answer, then read it
keyquorum --db mb.sqlite relay pull --output-dir ./answers
keyquorum --db mb.sqlite file open-answer --answer ./answers/<n>.kqpb \
  --slot ./usb=M.B --file report.txt.kqtf
```

`relay pull -import` lists a request or an answer and tells you which command opens it; it never imports one, because a request only asks.

14.8 History snapshots and the index

`file history` with `-export` (or `file history export -out`) writes a KQHS snapshot: the event history alone, which verifies on its own. `file verify-snapshot` with `-against` checks that a snapshot is a point in a file's history — for example, what a partner held when they last answered. A snapshot of an older point is still valid; it proves a prefix, not that nothing came after.

The store keeps `tracked_files`, `tracked_revisions` and `tracked_history_index` as a cache of metadata (no content, no trust decisions). `file list` reads it and `file reindex` rebuilds it from the containers, which remain the

authority.

14.9 Expiry and tombstones

`file expire` lets the scope owner (or one of its ancestors) schedule when a tracked file's content ends, or end it now. Because this cannot be undone, naming the owner is not enough: the command signs a challenge with `-slot` (or `-signing-key-file`) and checks it against the key your store has registered for that label. The key is opened only for that proof and released before anything changes. When the time has passed, the next command that needs the content destroys *every* retained revision's payload at once and records `FILE_EXPIRED` and `CONTENT_DESTROYED`. What remains is a tombstone: the revision graph, signatures and history still verify, no new revision can be added, and each later attempt to use the content is recorded as `EXPIRED_ACCESS_ATTEMPT` and refused. A container that is missing some payloads without a recorded destruction, or keeps any after one, fails verification.

```
keyquorum file expire report.txt.kqtf --as M.A --at "2026-12-31 17:00" \
--slot ./usb=M.A
keyquorum file expire report.txt.kqtf --as M.A --now --slot ./usb=M.A
```

Watch out

Expiry destroys the content of *this* container. Copies someone already holds — a letter they accepted, a checkout they wrote out — are their own files and are not reached.

14.10 Signed events

The hash chain shows that no earlier event was changed, removed or reordered. Commands that already prove you hold your key — `file rename`, `file expire` and `file resolve -reject` — also sign the events they append (over `KQ-FILE-HISTORY-EVENT-v1`). `file history` marks such an event [`signed by ...`] where your store's key for that person verifies it, and [`signature not verified in this store`] where it does not. Other events are protected by the chain alone and are never shown as signed. An event signature says who recorded the event; it decides nothing about whether a revision is trusted.

14.11 Recording what happens at a gate

`file link` ties a quorum-protected file (`-quorum-file`) or a password-locked file (`-locked-file`) to a tracked file. From then on the CLI appends what happened at that gate to the tracked file's history: each unlock attempt and its outcome (with the labels presented on a quorum success), each share link created, redeemed or revoked (the share's id, never its token), and the tombstone when the gate's own expiry destroys its ciphertext. The gates themselves are unchanged and never read the link; recording happens after the gate has answered and cannot change it. An unlock that fails before reaching the gate (for example while shares are being collected) is not recorded.

What is recorded is limited to what is safe to keep:

- **Quorum unlocks:** counts and policy words only — shares presented, threshold, distinct devices, minimum devices, custody mode, approval mode and parent approvals — never a share or key.
- **PIN checks:** only the outcome (`asked`, `verified`, `mismatch`, `locked` or `failed`); nothing when no PIN applies. Never the PIN, its hash or the attempt count.
- **Share links:** the share's id, never its token. A redemption is recorded as `redeemer=UNKNOWN_BEARER`, because holding a token proves nothing about who is holding it.

14.12 Limits this version states

Two limits are stated on purpose, so this version does not claim stronger guarantees than it has.

- **Historical authority is local key history.** The store preserves which identity and signing key each label held, as observed here. It does not establish complete generation-specific identity, relationship or reviewer authority.
- **Bridge disclosure awaits the owner's acceptance.** A `satisfied_by` of `BRIDGE` or `BRIDGE_AND_SCOPE_OWNER` reveals that a bridge approval took part in a decision. That stays in the portable history and in any export. Whether to keep it is the owner's policy decision.

The full architecture goes further than what is built. Today:

- Tracking starts with `file track` or the first `file sign` of a native file; nothing tracks a file on its own. A request for a file or a change is recorded only in a copy you name, and never as a delivery.
- A store keeps which topology generations it has held. Each command that resolves a file's topology generation (`track`, `check in`, `sign`, `merge` and the like, not a plain `verify`) also records which identity and signing key each label under that tree holds now, as the first and last generation it was observed in. A revision stamped with an older generation is also checked against every key recorded for that label and identity whose observed range covers it, in addition to the current key, so it keeps its meaning after the label is reissued or reassigned. This is the store's own key history, not a per-generation record of who held the label: a generation between two observations counts as covered, the current key is tried without proof it held the label then, a store that never saw a key has no record of it, and nothing proves when a signature was made.
- Conflicts are settled a whole revision at a time (`-keep` or `-from`), not hunk by hunk. The interactive review settles a conflict from the terminal only through the same commands as the command line, and a line has one recorded author in it: where several revisions on one side touched it, only the last is shown.
- Checkouts, signature checks and exports are recorded only when asked (`-record`), and only under the label `-as` names; that label is a claim, so the event is chained but never attested. Tampering is refused when a file is opened rather than recorded in it.
- Recording at gates is opt-in (`file link`) and best effort.

15 The mailbox relay

The hosted mailbox carries sealed `.kqpb` envelopes and public-tree slices. It indexes packages by the recipient fingerprint in the outer header and never unseals them; wrapped shares and private keys stay on the device. Your provider gives you a URL and an API key — you do not run the mailbox or mint keys yourself. Official `loadkey` / `relay` commands authenticate that host with a KeyQuorum-signed provider certificate before sending a bearer.

```
export KEYQUORUM_RELAY_URL=https://relay.example.com
# Load once; omit the key so it is prompted (stays out of shell history).
keyquorum loadkey --url https://relay.example.com
keyquorum relay push --dir ./bridge-packages
keyquorum relay pull --output-dir ./inbox
keyquorum --db alice.sqlite relay pull --import --share-file alice.key
# --api-key still works for scripts and is stored after a successful check.
keyquorum relay push --dir ./bridge-packages --api-key "$PUSH_KEY"
```

`loadkey` first verifies the relay's KeyQuorum-signed identity, then checks the key with the mailbox, then stores the key hash and a sealed copy of the bearer in the personal database. Later `relay` / `tree fetch` commands repeat that identity check and re-check the hash before using the key. An optional signed revocation list can be pointed at with `KEYQUORUM_PROVIDER_KRL`.

`relay push` also uploads every public tree in `-db` and `merges` it into the mailbox (unrelated nodes stay put). `relay pull` merges the returned slices into `-db` (then `-import` opens envelopes). `tree fetch` syncs topology without an envelope. Remote mailboxes must be `https://`; `http://` is accepted only for loopback.

15.1 The device mailbox

Device copy, move, and relocate (Section 12) use a second mailbox on the same host. `device.push` stores a sealed letter and publishes a device's public descriptor (device id, verify key, and slot public keys, signed by the device key). `device.pull` is bound to one recipient fingerprint, the same way `inbox.pull` is, and reads only that fingerprint's letters. A missing bearer is rejected, and a key of the wrong scope is rejected — the host does not mint keys over HTTP. Your provider issues `device.push` and `device.pull` the same way it issues `inbox` keys; `loadkey` stores them.

The host refuses a raw `KQTX` package and never unseals a letter. Bridge import does not read this mailbox, and device letters are not accepted into the bridge inbox.

15.2 Recovering from a lost key or a missed pull

If a key is lost or rotated, load the replacement your provider issues. Envelopes already delivered are unchanged.

```
keyquorum relay pull --after <id> --output-dir ./inbox
# or, to unseal them into --db directly:
# keyquorum relay pull --after <id> --import --share-file alice.key
```

replays anything not yet downloaded; **bridge private import** still rejects stale generations, so replaying an old envelope by mistake changes nothing.

16 Security model

This project handles cryptographic key material and encrypted user data directly. This section summarizes the trust boundaries described throughout the manual so they are visible in one place.

Security

Never commit private keys, tokens, secrets, API key bearers, provider certificates, revocation lists, device descriptors (**device.kq**, **device.skey**), slot tokens (***.kqst**), or plaintext copies of protected files to a repository. See **.gitignore** for the patterns already excluded. Compiling with **-features provider** does not make a host a trusted KeyQuorum provider.

16.1 What the mailbox can and cannot see

The hosted mailbox cannot decrypt **.kqpb** envelopes and must not be given wrapped shares or private keys. It stores the canonical *public* tree as JSON documents — labels, fingerprints, public keys, policy. Sending envelopes with **relay push** updates those documents from the sender's **-db**. Pulling returns a sliced copy for the pull-key fingerprint, which the CLI translates into local SQLite. Personal devices load a bearer with **keyquorum loadkey** (or **-api-key** once); they keep the hash and a sealed copy of the bearer in the owner-only org database.

Recover a lost bearer by loading the replacement your provider issues. Recover a missed update by pulling again and importing on the device that holds the matching decryption key.

16.2 Custody, quorum, and delegation, at a glance

- A key file with no container placement is its own device; distinct **-share-file** keys are distinct physical devices, and presenting the same key twice still counts once (Section 7, Section 3).
- Hardware custody requires each consumed Shamir share to sit on its own device id; logical custody lets several slots on one container meet a threshold together, but the container still counts as one device (Section 3).
- **unlock_approval = parent** layers a required parent signature on top of Shamir and the device count; it is opt-in, not a replacement for either (Section 3).
- A non-root **tree restructure** is a proposal until the parent countersigns; a direct tree update from that authorizer is refused; a routine employee reissue by the direct parent stays a single signature (Section 10, Section 9).
- A ghost identity keeps hierarchy and provenance but holds no usable private key: it cannot sign, satisfy a quorum, authorize an import, or be exported (Section 12).

16.3 Envelopes and authenticity

Every sealed letter in this manual — a private-bridge package, an authenticated update, an export bundle, a delivered file, a device transfer letter — shares the same shape: the outside names a recipient public key in the clear, the inside is sealed so only that recipient's private key can open it, and where the letter carries an authorization decision (an authenticated update, a delivery, a device transfer acknowledgement) it is additionally *signed* by whoever is authorizing that decision, checked against a key the opening store already has on file. A mailbox that carries these letters can route them by their outer fingerprint without ever being able to read or forge what is inside.

17 File formats quick reference

Every on-disk and on-wire artifact this manual mentions has its own magic bytes, so a store can tell at a glance what it is holding without trusting a file extension.

Magic	Extension	What it is
KQPB	.kqpb	Sealed PACKAGE envelope: private-bridge letters, authenticated updates, device transfer/relocate letters, file delivery letters and acks, tracked-file letters and acks.
KQXB	.kqxb	Sealed EXPORT_BUNDLE : a portable credential, locked-file or tracked-file export.
KQBS	.kqbs	Signature artifact from sign against a private bridge roster.
KQDV	device.kq	Signed device descriptor, verified against device.skey .
KQST	*.kqst	A slot token, sealing one logical identity to a device id.
KQTX	—	Device transfer package, signed by the source device, bound to a destination device id. Never stored raw on the mailbox.
KQTF	.kqtf	Tracked file: revisions, proofs, policy and hash-chained history (version 7: version 5 lets a payload be absent after expiry, 6 adds finalization proofs, 7 optional event signatures, each written only when present; versions 4 to 6 still open). Not a sealed envelope.
KQHS	.kqhs	History snapshot: a tracked file's event history alone, verifiable on its own and against its file.
—	.kqbn	Eviction notice — a public routing slip, not a sealed envelope.

Note

KQPB and KQXB share one sealed-envelope framing (magic, version, kind byte, recipient X25519 public key, sealed length); they are two *formats* of the same codec, not two independent designs. Device copy, move, and relocate letters are additional **PACKAGE** kind bytes under KQPB, not a separate format.

17.1 Roadmap

Private-key custody today is a key file (one key, one device) or a **keyquorum-device** container (one directory, many passphrase-wrapped slots). **generate** still prints a private key once and does not write it. Real USB token protocols are not implemented; a container is a directory, not an OS partition.

- **Persisted private-key custody** for **generate** beyond the key file and the slot token (for example, an OS keychain).
- **import** of password-vault / locked-file **export** bundles. The bundle format and encoder are already final; private-bridge **.kqpb** import is already implemented.

18 Command reference

The rest of this manual explains what each area of the CLI is for and how the pieces fit together; this section is the complete index of every **keyquorum** and **keyquorum-device** command and subcommand, so this manual alone — without a separate run of **-help** — says what the tool can do. Each row's description is the same one **-help** prints for that command. Flags are not reproduced here in full; run **keyquorum <command> -help** (or **keyquorum <command> <subcommand> -help**) for a command's complete flag list, defaults, and which flags conflict with or require others.

Note

keyquorum host is a hidden subcommand, compiled only with **-features provider**, and is intentionally omitted here: it is a build capability for operating a mailbox relay, not a customer-facing command, and is never documented outside the crate's own source.

18.1 Identity and hardware keys

Command	Purpose
<code>generate</code>	Generate an encryption or signing keypair. The private key prints to stdout once and is never written to disk by this tool; <code>-register</code> also registers the new public key.
<code>register</code>	Register an existing public key under a label.
<code>list</code>	List registered hardware keys and live split trees.
<code>revoke</code>	Revoke a registered key: drops its pairings and private-bridge membership everywhere; <code>-evict</code> also PSS-refreshes the surviving siblings of its leaf.
<code>remove</code>	Remove a registered hardware key.

18.2 Split-key trees

Command	Purpose
<code>split</code>	Split a secret into a live tree. <code>-leaf</code> builds the tree directly (siblings are bound automatically); <code>-tree-spec</code> is for restoring a nested snapshot.
<code>bind</code>	Pair two nodes (establish <code>node <-> peer</code>), or reseal a leaf onto a new public key.
<code>add</code>	Insert a leaf under a parent split and reshare that parent.
<code>tree</code>	Print every live tree, one tree by id, the lowest common ancestor of <code>-node</code> labels, or write the live spec as JSON.
<code>tree publish</code>	Upload a tree's public topology (no sealed shares) to the relay.
<code>tree restructure</code>	Announce a restructured tree: one authenticated envelope per active leaf at the next public generation. Unlike <code>publish</code> , recipients can verify who authorized the change.
<code>tree countersign</code>	Countersign a pending restructure proposal (key file, or <code>-device/-slot</code>).
<code>tree fetch</code>	Download the slice a pull key is allowed to see and merge it into this store.
<code>reconstruct</code>	Reconstruct a key's secret from raw shares (key files, container slots, or an interactive prompt).

18.3 Password vault

Command	Purpose
<code>vault add</code>	Store a new credential, optionally PIN-protected.
<code>vault get</code>	Retrieve a stored credential.

18.4 Bridges

Command	Purpose
<code>bridge allow</code>	Grant <code>-node</code> permission to form a cross-branch link with <code>-peer</code> .
<code>bridge deny</code>	Revoke that permission and drop any established link between them.
<code>bridge add</code>	Establish a pairing, if either node's whitelist allows it.
<code>bridge remove</code>	Tear down an established pairing; the whitelist entry is left intact.
<code>bridge list</code>	List whitelist entries and established pairings.
<code>bridge private create</code>	Create a private sign bridge: writes every recipient's <code>.kqpb</code> envelope first, then commits this store, so a failure at any point can simply be retried.
<code>bridge private list</code>	List private bridges, optionally for one split tree.
<code>bridge private show</code>	Show one private bridge by uid.
<code>bridge private events</code>	Print bridge-change events.
<code>bridge private import</code>	Import a per-person bridge package into this store.
<code>bridge private remove-member</code>	Drop a signing member: writes rotation/destroy packages first, then commits.

18.5 Authenticated updates

Command	Purpose
<code>reissue</code>	Announce a replaced hardware key: writes one authenticated envelope per store that holds the old key, so every recipient converges on the same token.
<code>updates</code>	Print the authenticated organization updates this store has applied.

18.6 Files and credentials

Command	Purpose
<code>access password</code>	Lock (<code>-state 0</code>) or unlock (<code>-state 1</code>) a password-protected file, optionally with a PIN and an expiry.
<code>access quorum</code>	Lock (<code>-state 0</code>) or unlock (<code>-state 1</code>) a hardware-key-quorum-protected file, or print its status (<code>-status</code>) — custody mode, minimum physical devices, unlock approval, and requirement tree.
<code>export credential</code>	Export a vault credential, sealed to a recipient's public key.
<code>export file</code>	Export a password-locked file, sealed to a recipient's public key.
<code>export tracked-file</code>	Export a complete, structurally verified <code>.kqtf</code> container, sealed to a recipient's public key.
<code>share create-credential</code>	Create a time-limited, revocable share link for a vault credential.
<code>share create-file</code>	Create a time-limited, revocable share link for a password-locked file.
<code>share redeem-credential</code>	Redeem a credential share token (prompted interactively, never as an argument).
<code>share redeem-file</code>	Redeem a file share token (prompted interactively, never as an argument).
<code>share revoke-credential</code>	Revoke a credential share.
<code>share revoke-file</code>	Revoke a file share.
<code>pin relock</code>	End a cached one-time PIN unlock window immediately.

18.7 Signing

Command	Purpose
<code>sign</code>	Sign a message with a private bridge plus a personal signing key.
<code>verify</code>	Verify an Ed25519 signature, standalone or against a private bridge's roster (<code>-bridge-uid</code>).

18.8 Delivery and the mailbox relay

Command	Purpose
<code>deliver send</code>	Seal a file to a registered label, signed with your signing key.
<code>deliver open</code>	Open a letter addressed to you: verify the sender, keep (or <code>-reject</code>) the file, and seal a signed acknowledgement back.
<code>deliver ack</code>	Check an acknowledgement sealed back to you.
<code>relay push</code>	Upload every <code>.kqpb</code> in a directory to the relay; also replaces public-tree documents from trees stored in <code>-db</code> .
<code>relay pull</code>	Download envelopes and the public-tree slice for a pull key; <code>-import</code> installs each envelope into <code>-db</code> directly.
<code>loadkey</code>	Check a relay API key with <code>POST /keycheck</code> and store it (and its hash) on this instance for later relay commands to reuse.

18.9 Tracked files

See section 14.

Command	Purpose
<code>file track</code>	Start tracking a file: make its first revision, sign it as its author, and write <code><file>.kqtf</code> . <code>-no-auto-merge</code> sends every fork to a person. <code>-owner-rule</code> , <code>-descendants-rule</code> , <code>-ancestors-rule</code> and <code>-cross-branch-rule</code> set what each role must present (<code>forbidden</code> , <code>author</code> , <code>author+parent</code> , or for cross-branch <code>author+bridge-or-owner</code> / <code>author+bridge+owner</code>); rules that cannot be met are refused.
<code>file policy</code>	Show the rules a tracked file's revisions are judged by and the policy hash every revision carries.
<code>file checkin</code>	Check in a new revision from a native file (signed with <code>-slot</code> , or <code>-unsigned</code>).
<code>file sign</code>	Sign a revision you authored (the current head unless <code>-revision</code>). Given a native file that is not tracked, this first content signature starts tracking it, exactly as <code>file track</code> does; the scope is your own label unless <code>-scope</code> names another.
<code>file rename</code>	Change the name a tracked file is shown under. The scope owner or an ancestor proves the label with <code>-slot</code> ; the file id and every revision id stay the same and the old and new names are recorded.
<code>file countersign</code>	Countersign a revision the author has signed.
<code>file finalize</code>	Mark a trusted revision final (scope owner or an ancestor, by slot). Trust itself is unchanged, and each store decides finality from its own keys.
<code>file send-history</code>	Seal this file's event history (a KQHS snapshot, no content) to a label, its root signed by you.
<code>file open-history</code>	Open a history snapshot letter and compare it with your copy (<code>-against</code>).
<code>file request</code>	Ask a holder for a file (<code>-file-id</code> and <code>-name</code>), or with <code>-change</code> and <code>-message</code> to change one you hold. Writes a sealed request (<code>-output-dir</code>) or uploads it (<code>-push</code>).
<code>file open-request</code>	Open a request made of you; <code>-file</code> records it once in your copy.

Command	Purpose
<code>file answer-request</code>	Accept or decline a request (<code>-decision</code>); <code>-file</code> records the answer once. An answer delivers nothing.
<code>file open-answer</code>	Open the holder's answer; <code>-file</code> records it against the request your copy shows you made.
<code>file bridge-approve</code>	As a member of a private bridge that reaches from the author to the scope, approve a revision with the bridge (kept in this store only).
<code>file resolve</code>	As a conflict's reviewer, keep one side (<code>-keep left right</code>) or supply the edited result (<code>-from</code>), signed as yours; <code>-reject</code> refuses a proposed merge.
<code>file merge</code>	Merge two divergent heads automatically when the edits allow it; otherwise record the conflict and who must review it, and on a <code>-features tui</code> build in an interactive terminal open the review. A merge is a new revision that stays untrusted until it is signed.
<code>file review</code>	Show what each side of a fork changed, and who reviews a conflict. <code>-interactive</code> needs a build with <code>-features tui</code> .
<code>file graph</code>	Show the revisions, their parents and their trust.
<code>file diff</code>	Show the changed lines between two revisions of text.
<code>file checkout</code>	Write a revision's native bytes to a new file; <code>-record</code> appends an event.
<code>file list</code>	List the tracked files this store has indexed (a cache; see <code>reindex</code>).
<code>file reindex</code>	Rebuild index rows from tracked files. Every file is verified first; if any fails, nothing changes. <code>-clear</code> also drops rows for files not named here.
<code>file status</code>	Show the heads, their trust, what would be shared, and the current and latest trusted revision ids.
<code>file history</code>	List the recorded events; <code>-export</code> also writes a KQHS snapshot.
<code>file history export</code>	The alternative spelling of <code>file history -export</code> : list the events and write a KQHS snapshot to <code>-out</code> .
<code>file history verify</code>	The alternative spelling of <code>file verify-snapshot</code> for a snapshot; given a tracked file it is <code>file verify</code> .
<code>file verify-snapshot</code>	Check a history snapshot; with <code>-against</code> , that it belongs to a file.
<code>file import</code>	Bring another copy of the same file's revisions and proofs into this one. A fork is kept as two heads; nothing is overwritten.
<code>file verify</code>	Verify the history chain and revision graph, and judge every revision.
<code>file expire</code>	Schedule when this file's content is destroyed, or destroy it now; needs <code>-slot</code> to prove you hold the owner label. Every retained revision's content goes at once; the history, the revision graph and every signature stay behind as a tombstone.
<code>file link</code>	Record what happens at a quorum-protected or password-locked file's gate in this tracked file's history. The gate is unchanged and never depends on it.
<code>file unlink</code>	Stop recording a gate in this tracked file's history.
<code>file share</code>	Seal the head (or <code>-revision</code>) to another label as a <code>.kqpb</code> letter, or its most recently stored trusted ancestor when it is not trusted. Trusted work on another branch is never substituted, and nothing is sent when no ancestor is trusted.
<code>file receive</code>	Open a tracked-file letter addressed to you. The sender is checked as the transport; the delivered revision is then judged by the file's own policy and accepted only if it is trusted here. Prints and records how the history compares with what this store accepted before (<code>FIRST</code> , <code>REPLAYED</code> , <code>NEWER</code> , <code>NOT_NEWER</code>); receiving the same letter again records nothing new.
<code>file ack</code>	Record an acknowledgement sealed back to you in your copy's history.

18.10 Device transfer

Command	Purpose
<code>transfer copy</code>	Copy an active key identity between two devices that are both open. The source stays active.
<code>transfer move</code>	Move an active key identity between two open devices. The source becomes a ghost only after the destination commits.
<code>transfer recover</code>	Finish or roll back a transfer that stopped between the two devices.
<code>transfer enroll</code>	Record a slot on this device as an active key identity.
<code>transfer list</code>	List identities on a device (ghosts hidden unless <code>-all</code>).
<code>transfer relay-send</code>	Seal a copy or move to the destination and leave the source prepared; the ghost applies only after the destination's acknowledgement is finalized.
<code>transfer relay-collect</code>	Pull a sealed copy or move, install it, and post the acknowledgement.
<code>transfer relay-finalize</code>	Apply a destination acknowledgement and finish the source.
<code>device init</code>	Create <code>device.kq</code> and an empty vault directory in this store's registry.
<code>device provision</code>	Add a passphrase-sealed slot to a container and print its public keys.
<code>device list</code>	List a container's slot labels and public keys.
<code>device bind</code>	Record a container's device id for a slot's registered keys.
<code>device register</code>	Register a slot's public key (a lone key file still uses top-level <code>register</code>).
<code>device publish</code>	Publish a container's public descriptor to the relay's device directory. Slot secrets stay local.
<code>device relay-relocate</code>	Seal one slot onto another device through the relay; prints the relocate id. The source slot stays until <code>relay-drop</code> sees the matching acknowledgement.
<code>device relay-accept</code>	Pull sealed relocates, install each slot, and post the acknowledgement.
<code>device relay-drop</code>	Delete the source slot after a verified relocate acknowledgement, keyed to the specific relocate id.

18.11 keyquorum-device: the container tool

A separate binary for one physical container of logical identity slots, used without an organization database (see `Devices` and `custody`). Private keys stay in the slot token; this tool never prints them.

Command	Purpose
<code>init</code>	Create <code>device.kq</code> and an empty vault directory.
<code>provision</code>	Add a slot; prints its public keys and prompts for a passphrase.
<code>list</code>	List slot labels and public keys from <code>device.kq</code> .
<code>public</code>	Print one slot's public keys.
<code>unwrap-share</code>	Unwrap a sealed share with a slot's encryption key; the raw share prints to stdout.
<code>sign</code>	Sign stdin with a slot's signing key; the hex signature prints to stdout.
<code>relocate</code>	Move a slot onto another container. The keypair is unchanged; only the device id it is bound to changes.

19 Contributing

AI coding agents working in this repository should read the relevant instructions file for their tool: `CLAUDE.md` for Claude, `AGENTS.md` for Codex and other agent tooling, and `.cursorrules` for Cursor.

20 License

Licensed under the MIT License. See `LICENSE` in the repository root for details.